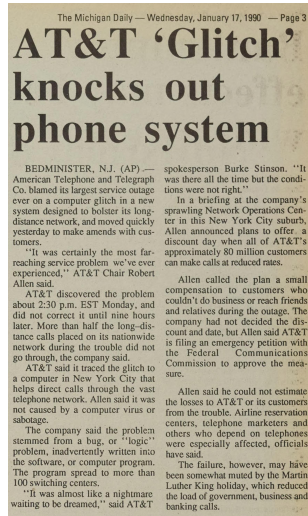


Abbildung: Auszug aus 'The Michigan Daily' vom 17.1.1990 [1]



Verifikation verteilter Systeme in Isabelle

Ein Vergleich von Kleppmann und I/O-Automaten

Sepp Stage

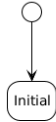
TU Chemnitz, Fakultät Betriebssysteme

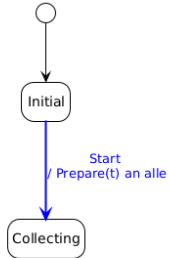


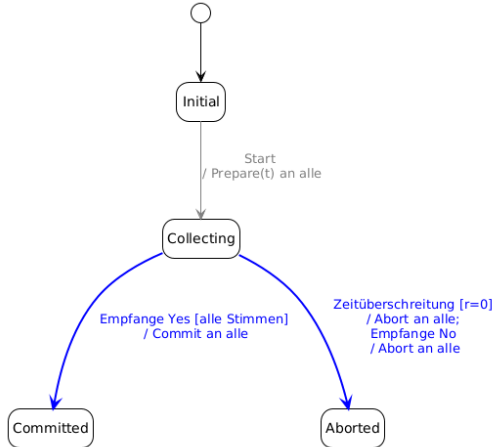
TECHNISCHE UNIVERSITÄT
CHEMNITZ

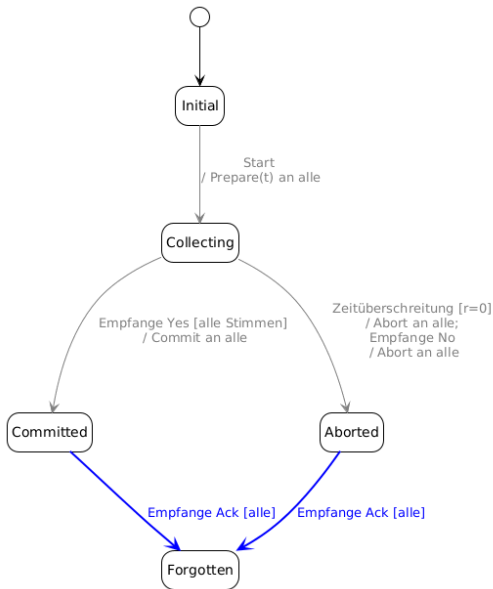
Inhaltsverzeichnis

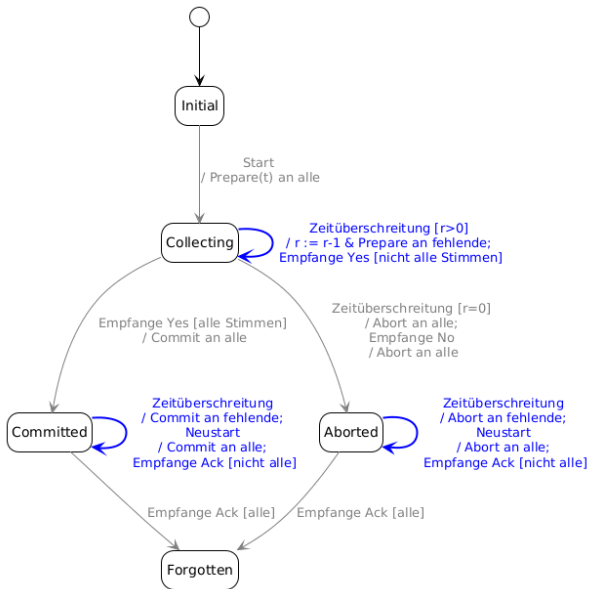
1. Das Zwei-Phase-Commit-Protokoll
2. I/O Automaten
3. Vergleich
4. Fazit
5. Quellen

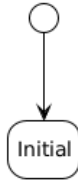


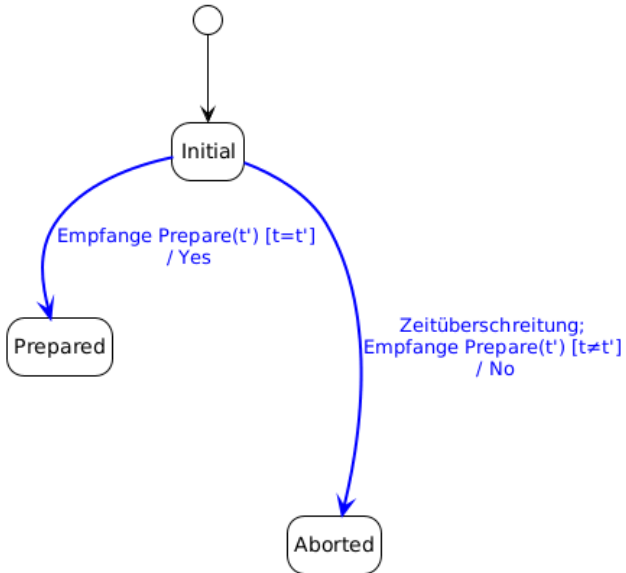


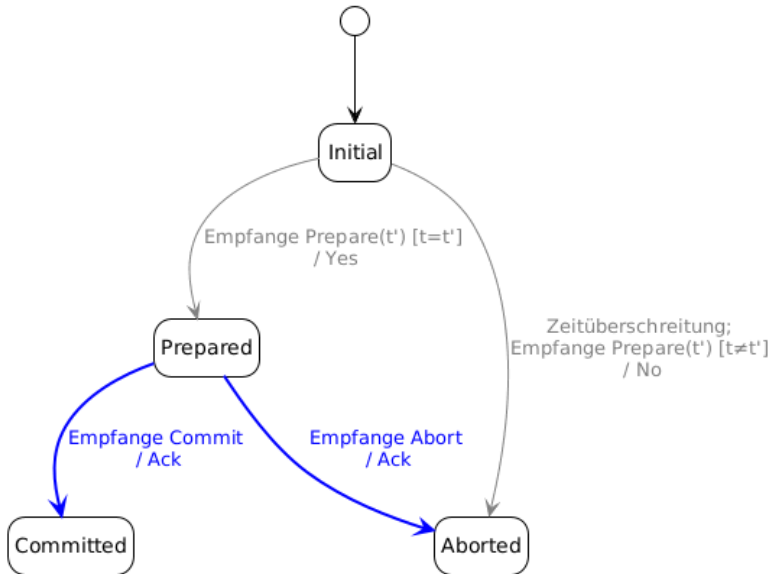


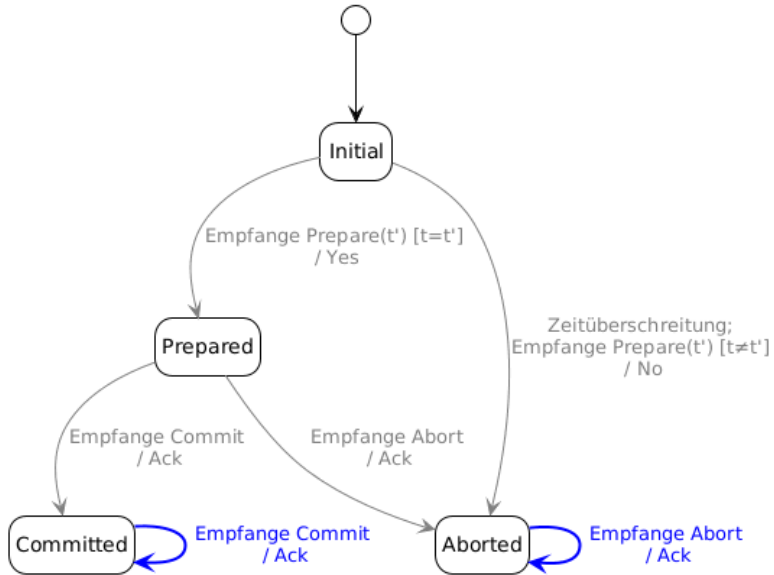








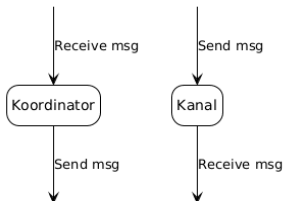




Definition einer Signatur S

- ▶ *Input-Übergänge* $in(S)$
- ▶ *Output-Übergänge* $out(S)$
- ▶ *Internal-Übergänge* $int(S)$

Abbildung: Simple I/O Automaten



Definition eines I/O Automaten A

- ▶ *Signatur* $sig(A)$
- ▶ *Menge an Zuständen* $states(A)$
- ▶ *nichtleere Menge an Startzuständen* $start(A) \subseteq states(A)$
- ▶ *Übergangsrelation*

$$trans(A) \subseteq states(A) \times acts(sig(A)) \times states(A)$$

- ▶ *Aufgabenpartition* $tasks(A)$

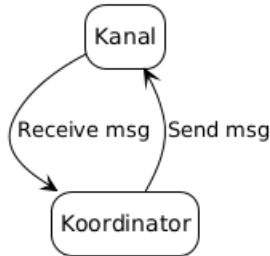
Ausführung eines I/O Automaten A

- ▶ *Ausführungsfragment* $s_0, \pi_1, s_1, \pi_2, \dots$
- ▶ $\forall i \geq 0. (s_i, \pi_{i+1}, s_{i+1}) \in \text{trans}(A)$
- ▶ *Ausführung falls* $s_0 \in \text{starts}(A)$

Komposition zweier I/O Automaten A & B

- ▶ Automaten müssen kompatibel sein
- ▶ Signaturen werden Verknüpft
- ▶ entstehende Zustände sind Vektoren (s_a, s_b) mit $s_i \in \text{start}(I)$
- ▶ A und B dürfen bei gemeinsamen Übergängen gleichzeitig agieren

Abbildung: Simple Komposition zweier I/O Automaten



Modellierung

- ▶ Nachrichten
- ▶ Events
- ▶ Validität von Events
- ▶ Zustände
- ▶ Übergangsfunktionen

```

5  datatype 't msg = Prepare
    ↳ (transaction: 't) | Yes |
    ↳ No | Commit | Abort | Ack

10 datatype ('proc, 'msg) event
11   = Start
12   | Receive (msg_sender:
    ↳ 'proc) (recv_msg: 'msg)
13   | Timeout
14   | Restart
  
```

- ▶ Nachrichten
- ▶ Signaturen
- ▶ Zustände
- ▶ Übergangsrelationen
- ▶ Automaten
- ▶ Komposition

- ▶ Nachrichten
- ▶ Events
- ▶ Validität von Events
- ▶ Zustände
- ▶ Übergangsfunktionen

```

7  datatype ('t, 'proc) state =
    ↳ Initial (transaction: 't)
    ↳ (all: "'proc set")
    ↳ (retry_count: "nat")
8  | Collecting (transaction:
    ↳ 't) (all: "'proc set")
    ↳ (yes: "'proc set")
    ↳ (retry_count: "nat")
9  | Prepared

```

...

- ▶ Nachrichten
- ▶ Signaturen
- ▶ Zustände
- ▶ Übergangsrelationen
- ▶ Automaten
- ▶ Komposition

- ▶ Nachrichten
 - ▶ Events
 - ▶ Validität von Events
 - ▶ Zustände
 - ▶ Übergangsfunktionen
-

```
47 fun tupac_step where
48   <tupac_step proc = (if proc
    ↪   = 0 then
    ↪   coordinator_step proc
    ↪   else participant_step)>
```

- ▶ Nachrichten
- ▶ Signaturen
- ▶ Zustände
- ▶ Übergangsrelationen
- ▶ Automaten
- ▶ Komposition

Flexibilität und Modularität

- ▶ Tendenziell monolithisch
 - ▶ gleicher Datentyp für Zustände
 - ▶ strenge Ausführungssemantik
 - ▶ fest vorgegebenes Netzwerk
-
- ▶ Komposition teil des Frameworks
 - ▶ eigener Datentyp für Zustände pro IOA
 - ▶ mehrere IOA können gleichzeitig Übergang verarbeiten
 - ▶ Netzwerk muss explizit modelliert werden

Beweisstruktur

► Invarianten

```

8  definition inv1 where
9    <inv1 msgs states  $\longleftrightarrow$ 
10      (( $\exists$ proc a ack'. proc  $\neq \emptyset \wedge$  states proc = Committed a
11         $\hookrightarrow$  ack')  $\longrightarrow$ 
          ( $\exists$ sender rcpt. (sender, Send rcpt Commit)
             $\hookrightarrow \in$  msgs))>

```

► Invarianten

```

6  definition inv1 where
7    <inv1 s  $\longleftrightarrow$ 
8      (( $\exists$ msgs p. (snd s) p = PState PCommitted msgs)  $\longrightarrow$ 
9        ( $\exists$ sender rcpt. (sender, rcpt, Commit)  $\in$ 
           $\hookrightarrow$  snd (fst s)))>

```


- Invarianten
- Induktion über Ausführungen (Invarianten)

```

90 lemma invariant1:
91   assumes <execute tupac_step (λp. Initial (init_val p)
      ↳ UNIV r) UNIV events msgs' states'>
92   shows "inv1 msgs' states'"
93   using assms proof(induction events arbitrary: msgs'
      ↳ states' r rule: List.rev_induct)
94   case Nil
102  case (snoc x events)
  
```

- Invarianten
- Induktion über Ausführungen (Invariante)

```

369 lemma invariant1:
370   "invariant (System t (UNIV - {0}) r) inv1"
371 proof(induction rule: invariantI)
372   case (1 s)
380   case (2 s t a)
  
```

- ▶ Invarianten
 - ▶ Induktion über Ausführungen (Invarianten)
 - ▶ Direkter Beweis der einheitlichen Zustimmung
-

- ▶ Invarianten
- ▶ Induktion über Ausführungen (Invariante)
- ▶ Direkter Beweis der einheitlichen Zustimmung

Abbildung: Auszug der schematischen Beweisstruktur für invariant1_trans

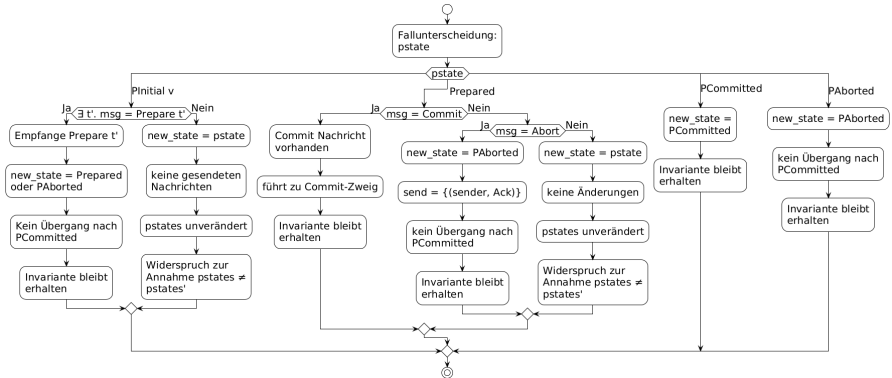
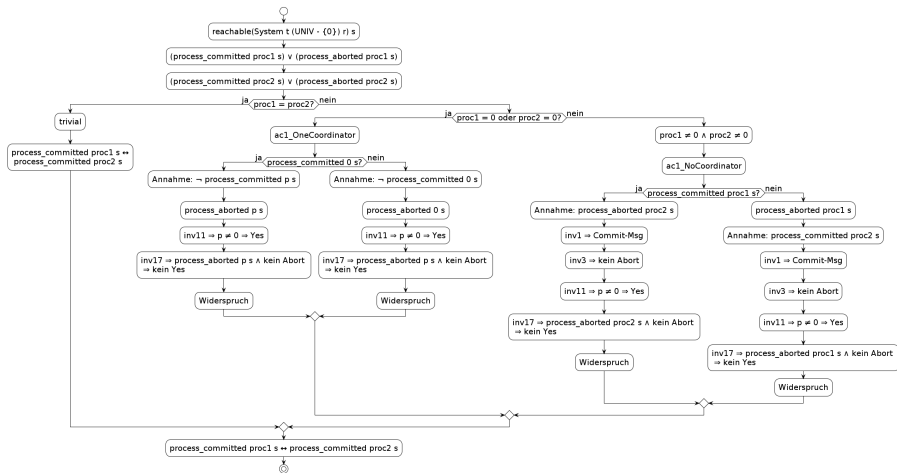


Abbildung: Schematische Beweisstruktur für ac1_Theorem



- ▶ starre Struktur

- ▶ wenig Struktur
- ▶ Netzwerk IOA muss betrachtet werden
- ▶ Nicht-Determinismus

Nachweis von Liveness-Eigenschaften

- ▶ Netzwerk mit wenig Garantien
- ▶ bewusst weggelassen
- ▶ 'leicht' erweiterbar

-
- ▶ Liveness nativ vorgesehen
 - ▶ Netzwerk Garantien können selbst festgelegt werden

Automatisierungsgrad in Isabelle

- ▶ wenig Hilfslemmas
- ▶ kompaktes Framework

-
- ▶ viele Definitionen
 - ▶ Aussagen werden größer

```

373 moreover have "starts_of (System t (UNIV - {0}) r) =
    ↪  {((CState (CInitial (t 0) UNIV r) {}, {}), λp. PState
    ↪  (PInitial (t p)) {}))}"
374 unfolding starts_of_def System_def par_def
    ↪  Coordinator_def Channel_def Participants_def by auto
  
```

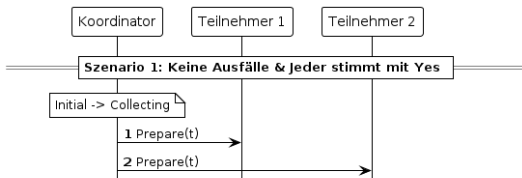
Framework Dokumentation

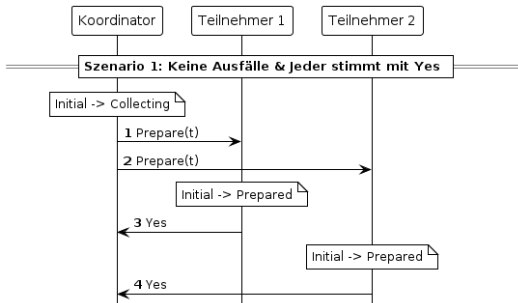
- ▶ 2h Präsentation zu Konsens Algorithmus
- ▶ Code online
- ▶ Blogbeitrag zu Framework

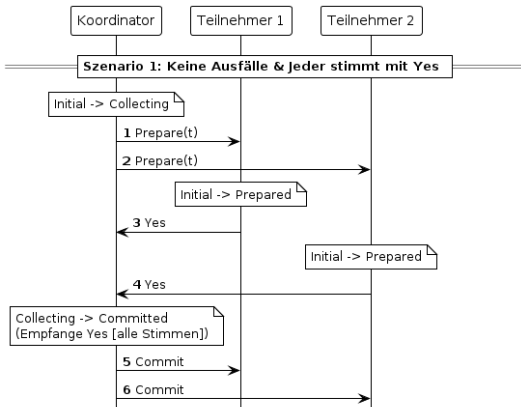
-
- ▶ mathematisches Modell verbreitet
 - ▶ Beispiele mit wenig Code

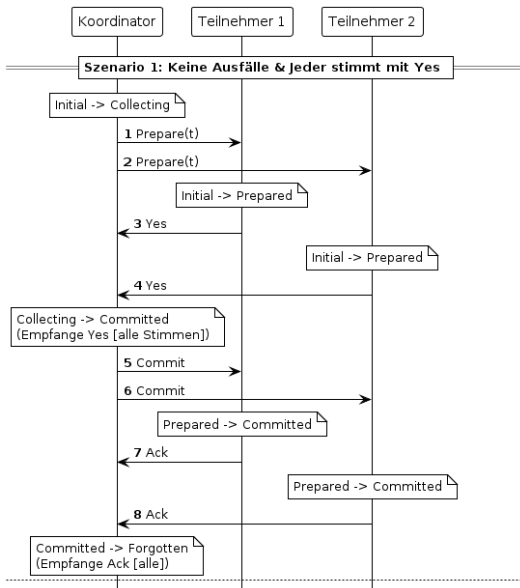
"By itself, the I/O automaton model has very little structure, which allows it to be used for modelling many different types of distributed systems. Additional structure must be added to the basic model to enable it to describe particular types of asynchronous systems."
~Lynch, Distributed Algorithms S.199 [2]

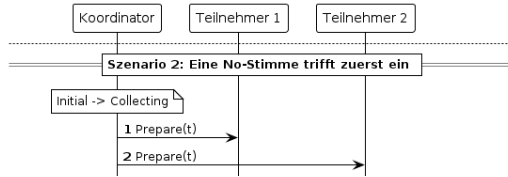
- [1] *January 17, 1990 (vol. 100, iss. 73) - Image 3.* URL: <https://digital.bentley.umich.edu/midaily/mdp.39015071754852/65>.
- [2] Nancy A. Lynch. *Distributed Algorithms*. 16. Apr. 1996. URL: <https://archive.org/details/distributedalgor0000lync/page/208/mode/2up> (besucht am 06. 04. 2026).
- [3] Jens Lechtenbörger. *2-Phase-Commit Protocol*. type. Universität Münster. URL: <http://www.lgis.informatik.uni-kl.de/cms/fileadmin/courses/SS2008/Informationssysteme/Addons/2PC.pdf> (besucht am 04. 04. 2026).
- [4] Tobias Nipkow Olaf Müller Konrad Slind. *Theory Automata*. URL: <https://isabelle.in.tum.de/library/HOL/IOA/Automata.html> (besucht am 04. 12. 2026).

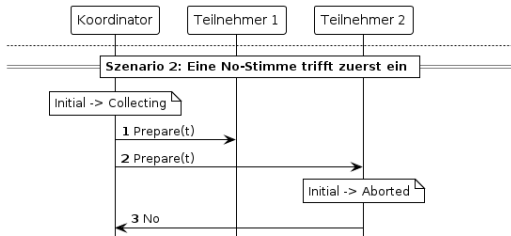


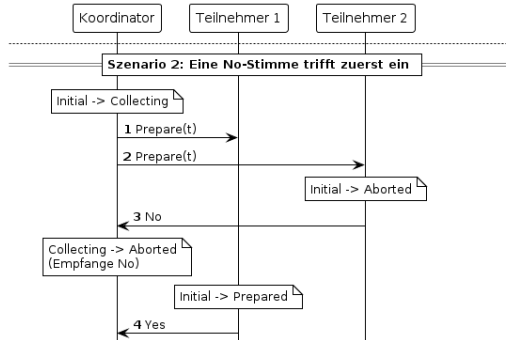


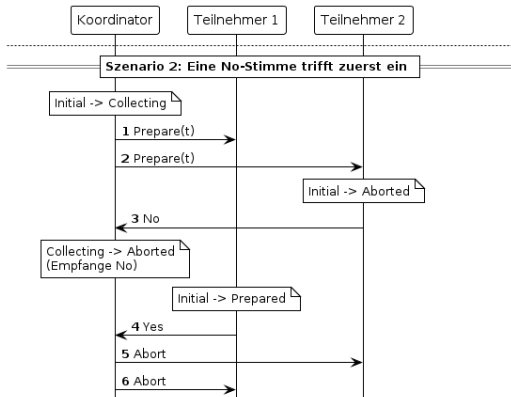


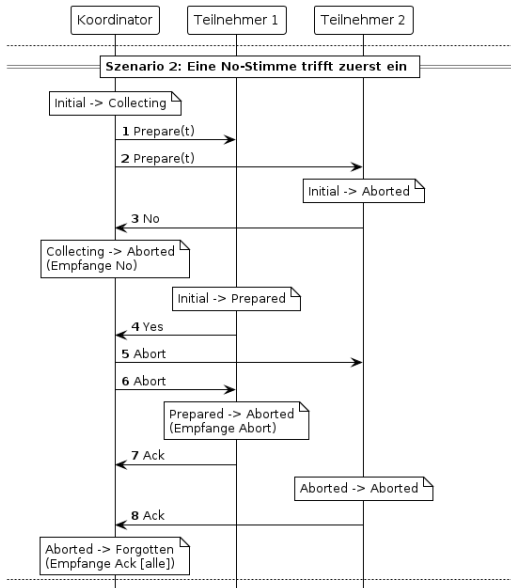


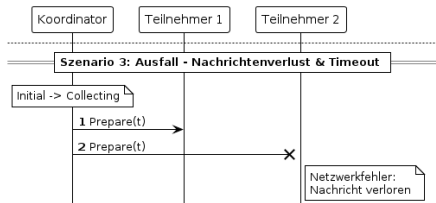


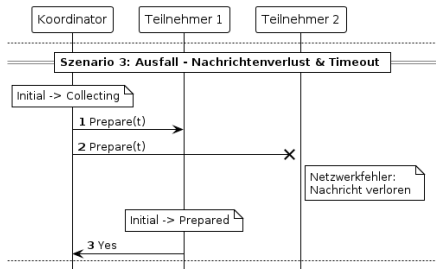


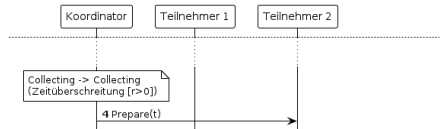
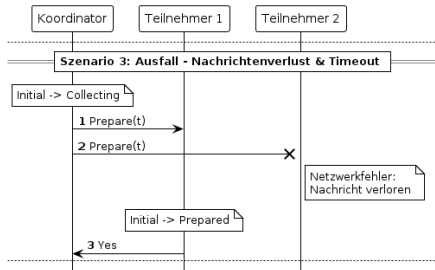


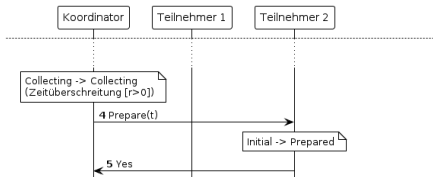
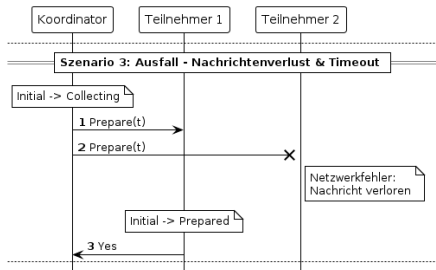


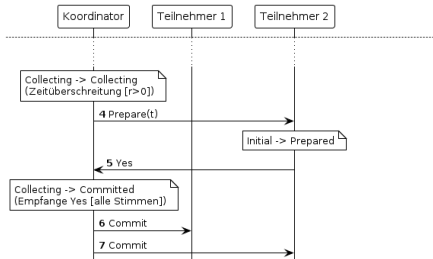
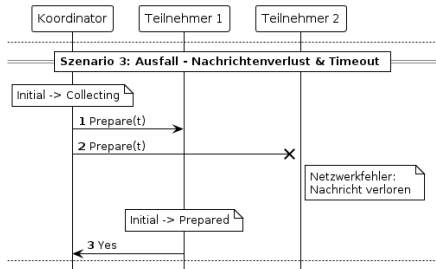


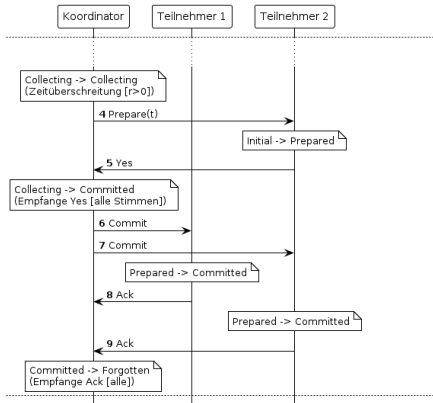
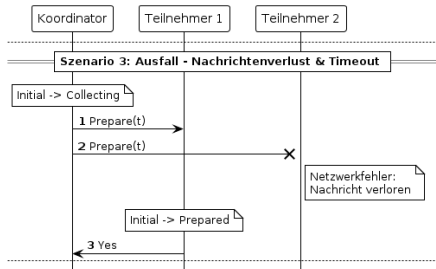


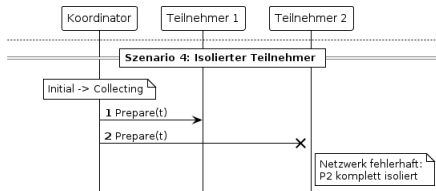


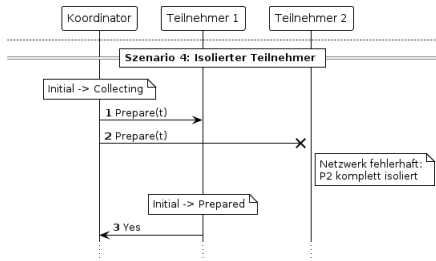


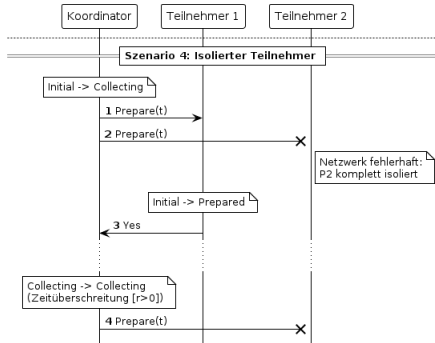


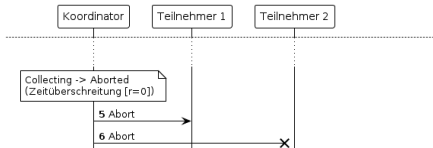
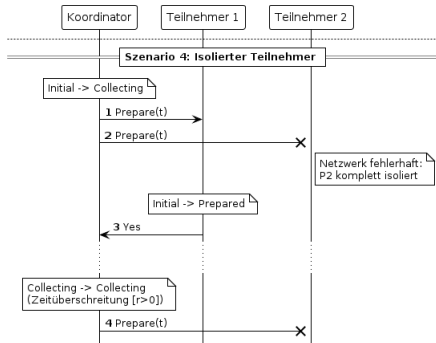


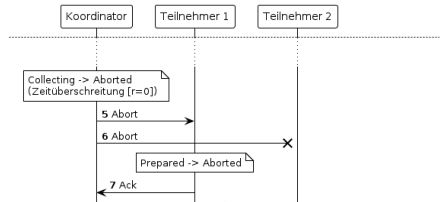
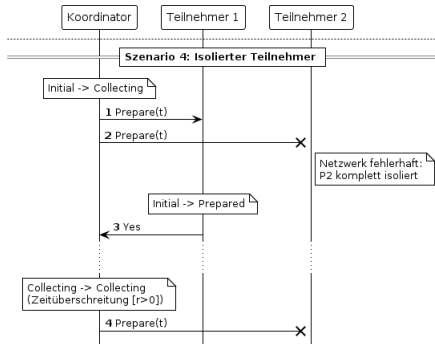


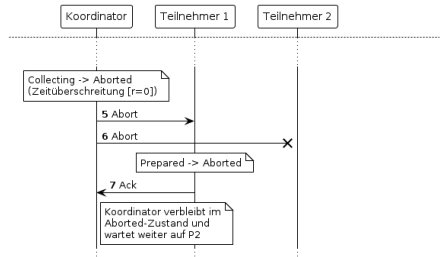
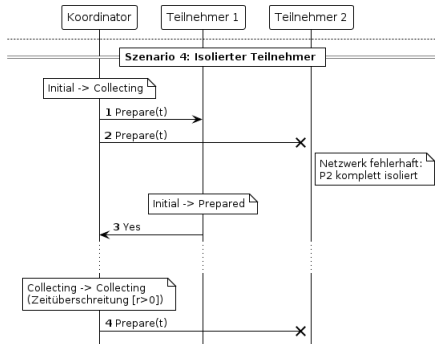


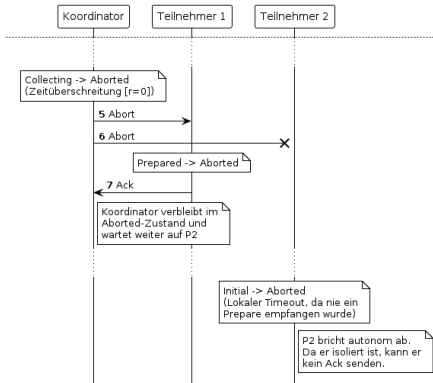
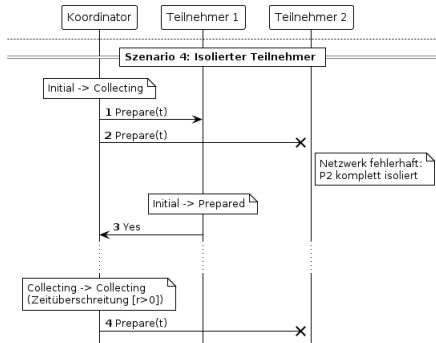












```

5  datatype 't msg = Prepare (transaction: 't) | Yes | No |
   ↳ Commit | Abort | Ack
10  datatype ('proc, 'msg) event
11    = Start
12    | Receive (msg_sender: 'proc) (recv_msg: 'msg)
13    | Timeout
14    | Restart
  
```

```

6  datatype 't msg = Prepare (transaction: 't) | Yes | No |
   ↳ Commit | Abort | Ack
8  datatype ('proc, 'msg) action =
9    Start "'proc"
10   | Send "'proc" "'proc" "'msg"
11   | Receive "'proc" "'proc" "'msg"
12   | Timeout "'proc"
13   | Restart "'proc"
  
```

```

50 definition coord_asig :: "'proc  $\Rightarrow$  ('proc,'t) action
     $\hookrightarrow$  signature" where
51 "coord_asig pid =
52   ({Receive q pid m | q m. True}  $\cup$  {Start pid, Timeout pid,
     $\hookrightarrow$  Restart pid},
53    {Send pid q m | q m. True},
54    {})"
89 definition parts_asig :: "'proc set  $\Rightarrow$  ('proc,'t msg) action
     $\hookrightarrow$  signature" where
90 "parts_asig P =
91   ({Receive q p m | q p m. p  $\in$  P}  $\cup$  {Timeout p | p. p  $\in$  P}  $\cup$ 
     $\hookrightarrow$  {Restart p | p. p  $\in$  P},
92    {Send p q m | p q m. p  $\in$  P},
93    {})"
113 definition chan_asig :: "('proc,'t) action signature" where
114 "chan_asig =
115   ({Send s r m | s r m. True},
116    {Receive s r m | s r m. True},
117    {})"
  
```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

17 datatype ('t, 'proc) cstate = CInitial (transaction: 't)
   ↪  (all: "'proc set") (retry_count:"nat")
18 | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
19 | CCommitted (all: "'proc set") (ack: "'proc set")
20 | CAborted (all: "'proc set") (ack: "'proc set")
21 | Forgotten
22 datatype ('t, 'proc) coord_state = CState "('t, 'proc) cstate"
   ↪  "('proc × 'proc × 't msg) set"
  
```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:""'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

61 datatype 't pstate = PInitial (transaction: 't)
62 | Prepared
63 | PCommitted
64 | PAborted
65 datatype ('t, 'proc) part_state = PState "'t pstate" "('proc
   ↪  × 'proc × 't msg) set"
  
```



```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

106 type_synonym ('proc, 't) chan_state = "('proc × 'proc × 't
   ↪  msg) set"
  
```

```

14 fun coordinator_step::
15   "'proc ⇒ ('t, 'proc) state ⇒ ('proc, 't msg) event ⇒
    ⇨ ('t, 'proc) state × ('proc, 't msg) send set" where
  
```

```

34 fun participant_step::
35   "'('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t,
    ⇨ 'proc) state × ('proc, 't msg) send set" where
  
```

```

24 fun coordinator_step::
25   "'proc ⇒ ('t, 'proc) cstate ⇒ ('proc, 't msg) action ⇒
    ⇨ ('t, 'proc) cstate × ('proc × 't msg) set" where
  
```

```

67 fun participant_step::
68   "'proc ⇒ 't pstate ⇒ ('proc, 't msg) action ⇒ 't pstate
    ⇨ × ('proc × 't msg) set" where
  
```

```

16  <coordinator_step pid (Initial t a r) Start = (Collecting t
    ↪  a {pid} r, {Send p (Prepare t) | p. p ∈ a})> |
    _____
26  <coordinator_step pid (CInitial t a r) (Start _) =
    ↪  (Collecting t a {pid} r, {(q, (Prepare t)) | q. q ∈ a -
    ↪  {pid}})> |
  
```

```

17  <coordinator_step pid (Collecting t a y r) (Receive sender
    ↪  msg) =
18    (case msg of
19      Yes ⇒(if  $y \cup \{\text{sender}\} = a$  then (Committed a {pid},
        ↪  {Send p (Commit) |  $p.p \in a$ }) else (Collecting t
        ↪  a ( $y \cup \{\text{sender}\}$ ) r, {}))) |
20      No ⇒(Aborted a {pid}, {Send p (Abort) |  $p.p \in a$ }) |
21      _ ⇒(Collecting t a y r, {}))> |
  
```

```

27  <coordinator_step pid (Collecting t a y r) (Receive sender
    ↪  _ msg) =
28    (case msg of
29      Yes ⇒(if  $y \cup \{\text{sender}\} = a$  then (CCommitted a {pid},
        ↪  {(q, Commit) |  $q.q \in a - \{\text{pid}\}$ }) else
        ↪  (Collecting t a ( $y \cup \{\text{sender}\}$ ) r, {}))) |
30      No ⇒(CAborted a {pid}, {(q, Abort) |  $q.q \in a -$ 
        ↪  {pid}}) |
31      _ ⇒(Collecting t a y r, {}))> |
  
```

```

22  <coordinator_step pid (Collecting t a y 0) Timeout =
    ↪ (Aborted a {pid}, {Send p (Abort) | p. p ∈ a})> |
23  <coordinator_step _ (Collecting t a y (Suc r)) Timeout =
    ↪ (Collecting t a y r, {Send p (Prepare t) | p. p ∈ (a -
    ↪ y)})> |

```

```

32  <coordinator_step pid (Collecting t a y 0) (Timeout _) =
    ↪ (CAborted a {pid}, {(q, Abort) | q. q ∈ a - {pid}})> |
33  <coordinator_step pid (Collecting t a y (Suc r)) (Timeout
    ↪ _ ) = (Collecting t a y r, {(q, Prepare t) | q. q ∈ (a -
    ↪ y)})> |

```

```

26  <coordinator_step _ (Committed a ack') Timeout = (Committed
    ↪  a ack', {Send p Commit | p. p ∈ (a - ack')})> |
27  <coordinator_step _ (Committed a ack') Restart = (Committed
    ↪  a ack', {Send p Commit | p. p ∈ a})> |

36  <coordinator_step pid (CCommitted a ack') (Timeout _) =
    ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ (a - ack')})>
    ↪  |
37  <coordinator_step pid (CCommitted a ack') (Restart _) =
    ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ a - {pid}})>
    ↪  |
  
```

```

44 definition coord_trans ::
45   "'proc  $\Rightarrow$  (('t,'proc) coord_state  $\times$  ('proc,'t msg) action  $\times$ 
     $\hookrightarrow$  ('t,'proc) coord_state) set" where
46   "coord_trans pid =
47     {(CState s msgs, a, CState s' (msgs  $\cup$  (( $\lambda$ msg. (pid, msg)) `
     $\hookrightarrow$  sent)))) | s a s' msgs sent. coordinator_step pid s a =
     $\hookrightarrow$  (s', sent)  $\wedge$  ( $\exists$ s m. a = Start pid  $\vee$  a = Receive s pid m
     $\hookrightarrow$   $\vee$  a = Timeout pid  $\vee$  a = Restart pid)}
48    $\cup$  {(CState s msgs, Send pid rcpt msg, CState s (msgs -
     $\hookrightarrow$  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg)  $\in$ 
     $\hookrightarrow$  msgs}"

56 definition Coordinator :: "'p  $\Rightarrow$  't  $\Rightarrow$  'p set  $\Rightarrow$  nat  $\Rightarrow$ 
     $\hookrightarrow$  (('p,'t msg) action, ('t,'p) coord_state) ioa" where
57   "Coordinator pid t a r = (coord_asig pid, {CState (CInitial t
     $\hookrightarrow$  a r) {}}, coord_trans pid, {}, {})"
  
```

```

81 definition part_trans ::
82   "'proc  $\Rightarrow$  (('t,'proc) part_state  $\times$  ('proc,'t msg) action  $\times$ 
       $\hookrightarrow$  ('t,'proc) part_state) set" where
83   "part_trans pid =
84     {(PState s msgs, a, PState s' (msgs  $\cup$  (( $\lambda$ msg. (pid, msg))  $\cdot$ 
       $\hookrightarrow$  sent))) | s a s' msgs sent. participant_step pid s a =
       $\hookrightarrow$  (s', sent)  $\wedge$  ( $\exists$ s m. a = Receive s pid m  $\vee$  a = Timeout
       $\hookrightarrow$  pid  $\vee$  a = Restart pid)}
85    $\cup$  {(PState s msgs, Send pid rcpt msg, PState s (msgs -
       $\hookrightarrow$  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg)  $\in$ 
       $\hookrightarrow$  msgs}"

```



```

95 definition parts_trans :: "'proc set  $\Rightarrow$  (('proc  $\Rightarrow$  ('t, 'proc)
     $\hookrightarrow$  part_state)  $\times$  ('proc, 't msg) action  $\times$  ('proc  $\Rightarrow$ 
     $\hookrightarrow$  ('t, 'proc) part_state)) set" where
96 "parts_trans P = { (s, a, s') .
97    $\forall p$ . if  $p \in P \wedge ((\exists q m. a = \text{Receive } q \ p \ m) \vee a = \text{Timeout } p$ 
     $\hookrightarrow \vee a = \text{Restart } p \vee (\exists q m. a = \text{Send } p \ q \ m))$ 
98   then  $(s \ p, a, s' \ p) \in \text{part\_trans } p$ 
99   else  $s' \ p = s \ p \}$ "

101 definition Participants :: "'proc set  $\Rightarrow$  ('proc  $\Rightarrow$  't)  $\Rightarrow$ 
     $\hookrightarrow$  (('proc, 't msg) action, 'proc  $\Rightarrow$  ('t, 'proc) part_state)
     $\hookrightarrow$  ioa" where
102 "Participants P t = (parts_asig P,  $\{\lambda p. \text{PState } (\text{PInitial } (t$ 
     $\hookrightarrow p)) \{\}\}$ , parts_trans P,  $\{\}$ ,  $\{\}$ )"
  
```

```

108 definition chan_trans :: "('proc,'t) chan_state × ('proc,'t
    ↪ msg) action × ('proc,'t) chan_state) set" where
109 "chan_trans =
110   {(M, Send s r m, M ∪ {(s,r,m)}) | M s r m. True}
111   ∪ {(M, Receive s r m, M) | M s r m. (s,r,m) ∈ M}"
112
113 definition Channel :: "('proc,'t msg) action, ('proc,'t)
    ↪ chan_state) ioa" where
114 "Channel = (chan_asig, { {} }, chan_trans, {}, {})"
  
```

```
47 fun tupac_step where
48   <tupac_step proc = (if proc = 0 then coordinator_step proc
    ↪   else participant_step)>
```

```
124 definition System where
125   "System t P r = ((Coordinator 0 (t 0) (P ∪ {0}) r || Channel)
    ↪   || Participants P t)"
```

Wenn es einen Teilnehmer im Committed-Zustand gibt, so muss es eine Commit-Nachricht geben

```

8  definition inv1 where
9     $\langle \text{inv1 msgs states} \longleftrightarrow$ 
10       $((\exists \text{proc } a \text{ ack}'. \text{proc} \neq \emptyset \wedge \text{states proc} = \text{Committed } a$ 
11         $\hookrightarrow \text{ack}') \longrightarrow$ 
           $(\exists \text{sender rcpt. (sender, Send rcpt Commit)} \in$ 
             $\hookrightarrow \text{msgs})) \rangle$ 

```

```

6  definition inv1 where
7     $\langle \text{inv1 } s \longleftrightarrow$ 
8       $((\exists \text{msgs } p. (\text{snd } s) p = \text{PState PCommitted msgs}) \longrightarrow$ 
9         $(\exists \text{sender rcpt. (sender, rcpt, Commit)} \in \text{snd}$ 
           $\hookrightarrow (\text{fst } s))) \rangle$ 

```

Wenn es eine Commit-Nachricht gibt oder der Koordinator im Committed-Zustand ist, so hat jeder Teilnehmer eine Yes-Nachricht gesendet

```

13 definition inv11 where
14    $\langle \text{inv11 msgs states procs} \longleftrightarrow$ 
15      $((\exists \text{sender rcpt. } (\text{sender, Send rcpt Commit}) \in \text{msgs}) \vee$ 
16        $\hookrightarrow (\exists \text{all ack. states } \emptyset = \text{Committed all ack}) \longrightarrow$ 
17          $(\forall p \in \text{procs. } p \neq \emptyset \longrightarrow (\exists \text{rcpt. } (p, \text{Send rcpt}$ 
18            $\hookrightarrow \text{Yes}) \in \text{msgs}))) \rangle$ 

```

```

11 definition inv11 where
12    $\langle \text{inv11 procs s} \longleftrightarrow$ 
13      $((\exists \text{sender rcpt. } (\text{sender, rcpt, Commit}) \in \text{snd (fst s)}) \vee$ 
14        $\hookrightarrow (\exists \text{all ack msgs. fst (fst s) = CState (CCommitted all}$ 
15          $\hookrightarrow \text{ack) msgs}) \longrightarrow$ 
16          $(\forall p \in \text{procs. } p \neq \emptyset \longrightarrow (\exists \text{rcpt. } (p, \text{rcpt,}$ 
17            $\hookrightarrow \text{Yes}) \in \text{snd (fst s)}))) \rangle$ 

```

Wenn der Koordinator im Collecting-Zustand ist, so stimmen hat er sich die korrekten Werte abgespeichert

```

22  definition inv13 where
23    <inv13 msgs states procs  $\longleftrightarrow$ 
24      ( $\forall t$  all yes r. states  $\emptyset = \text{Collecting } t$  all yes r  $\longrightarrow$ 
25        (procs = all  $\wedge (\forall p \in \text{yes}. p \neq \emptyset \longrightarrow (\exists \text{rcpt.}$ 
            $\hookrightarrow (p, \text{Send rcpt Yes}) \in \text{msgs}))))$ >
  
```

```

16  definition inv13 where
17    <inv13 procs s  $\longleftrightarrow$ 
18      ( $\forall t$  all yes r msgs. fst (fst s) = CState (Collecting t
19         $\hookrightarrow$  all yes r) msgs  $\longrightarrow$ 
           (procs = all  $\wedge (\forall p \in \text{yes}. p \neq \emptyset \longrightarrow (\exists \text{rcpt.}$ 
            $\hookrightarrow (p, \text{rcpt, Yes}) \in \text{snd (fst s)}))))$ >
  
```

Wenn der Koordinator im Initial-Zustand ist, so stimmen hat er sich die korrekte Menge an Prozessen abgespeichert

```

27 definition inv14 where
28    $\langle \text{inv14 msgs states procs} \longleftrightarrow$ 
29      $(\forall t \text{ all } r. \text{ states } \emptyset = \text{Initial } t \text{ all } r \longrightarrow \text{procs} = \text{all}) \rangle$ 

```

```

21 definition inv14 where
22    $\langle \text{inv14 procs s} \longleftrightarrow$ 
23      $(\forall t \text{ all } r \text{ msgs. fst (fst s) = CState (CInitial } t \text{ all } r)$ 
       $\hookrightarrow \text{msgs} \longrightarrow \text{procs} = \text{all}) \rangle$ 

```

Wenn sich der Koordinator im Initial-, Collecting- oder Committed- Zustand befindet, so wurde keine Abort-Nachricht gesendet

```

31 definition inv15 where
32    $\langle \text{inv15 msgs states} \longleftrightarrow$ 
33      $((\exists t \text{ all yes } r. \text{ states } \emptyset = \text{Initial } t \text{ all } r \vee \text{ states } \emptyset =$ 
34        $\hookrightarrow \text{Collecting } t \text{ all yes } r \vee \text{ states } \emptyset = \text{Committed all}$ 
35        $\hookrightarrow \text{yes}) \longrightarrow$ 
36        $(\forall \text{sender rcpt. } (\text{sender, Send rcpt Abort}) \notin$ 
37        $\hookrightarrow \text{msgs})) \rangle$ 

```

```

25 definition inv15 where
26    $\langle \text{inv15 s} \longleftrightarrow$ 
27      $((\exists t \text{ all yes } r \text{ msgs. fst (fst s) = CState (CInitial } t \text{ all}$ 
28        $\hookrightarrow r) \text{ msgs} \vee \text{fst (fst s) = CState (Collecting } t \text{ all yes}$ 
29        $\hookrightarrow r) \text{ msgs} \vee \text{fst (fst s) = CState (CCommitted all yes)}$ 
30        $\hookrightarrow \text{msgs}) \longrightarrow$ 
31        $(\forall \text{sender rcpt. } (\text{sender, rcpt, Abort}) \notin \text{snd (fst}$ 
32        $\hookrightarrow s))) \rangle$ 

```


Wenn sich ein Teilnehmer im Aborted-Zustand befindet und keine Abort-Nachricht gesendet wurde, so hat der Teilnehmer auch keine Yes-Nachricht gesendet

```

40 definition inv17 where
41   <inv17 msgs states  $\longleftrightarrow$ 
42     ( $\forall p. p \neq 0 \wedge (\exists \text{all ack. states } p = \text{Aborted all ack}) \wedge$ 
        $\hookrightarrow (\forall \text{sender rcpt. (sender, Send rcpt Abort)} \notin \text{msgs})$ 
        $\hookrightarrow \longrightarrow (\forall \text{rcpt. (p, Send rcpt Yes)} \notin \text{msgs}))$ >
  
```

```

30 definition inv17 where
31   <inv17 s  $\longleftrightarrow$ 
32     ( $\forall p. p \neq 0 \wedge (\exists \text{msgs. (snd s) } p = \text{PState PAborted msgs}) \wedge$ 
        $\hookrightarrow (\forall \text{sender rcpt. (sender, rcpt, Abort)} \notin \text{snd (fst$ 
        $\hookrightarrow \text{s)}) \longrightarrow (\forall \text{rcpt. (p, rcpt, Yes)} \notin \text{snd (fst s)}))$ >
  
```

Wenn sich der Koordinator im Initial-, Collecting- oder Aborted- Zustand befindet, so wurde keine Commit-Nachricht gesendet

```

52  definition inv110 where
53    <inv110 msgs states  $\longleftrightarrow$ 
54      (( $\exists t$  all yes r. states  $\emptyset = \text{Initial } t$  all r  $\vee$  states  $\emptyset =$ 
         $\hookrightarrow$  Collecting t all yes r  $\vee$  states  $\emptyset = \text{Aborted all yes}$ )
         $\hookrightarrow \longrightarrow$ 
55        ( $\forall \text{sender rcpt. (sender, Send rcpt Commit)} \notin$ 
           $\hookrightarrow \text{msgs}))$ >
  
```

```

34  definition inv110 where
35    <inv110 s  $\longleftrightarrow$ 
36      (( $\exists t$  all yes r msgs. fst (fst s) = CState (CInitial t all
         $\hookrightarrow$  r) msgs  $\vee$  fst (fst s) = CState (Collecting t all yes
         $\hookrightarrow$  r) msgs  $\vee$  fst (fst s) = CState (CAborted all yes)
         $\hookrightarrow$  msgs)  $\longrightarrow$ 
37        ( $\forall \text{sender rcpt. (sender, rcpt, Commit)} \notin \text{snd}$ 
           $\hookrightarrow \text{(fst s)}))$ >
  
```

Wenn eine Commit-Nachricht gesendet wurde, so befindet sich der Koordinator entweder im Committed- oder Vergessen-Zustand

```

57 definition inv111 where
58   <inv111 msgs states  $\longleftrightarrow$ 
59     (( $\exists$ sender rcpt. (sender, Send rcpt Commit)  $\in$  msgs)  $\longrightarrow$ 
       $\hookrightarrow$  ( $\exists$ all ack. states  $\emptyset$  = Committed all ack  $\vee$  states  $\emptyset$  =
       $\hookrightarrow$  Vergessen)))>

```

```

39 definition inv111 where
40   <inv111 s  $\longleftrightarrow$ 
41     (( $\exists$ sender rcpt. (sender, rcpt, Commit)  $\in$  snd (fst s))  $\longrightarrow$ 
42       ( $\exists$ all ack cmsgs. fst (fst s) = CState (CCommitted all
       $\hookrightarrow$  ack) cmsgs  $\vee$  fst (fst s) = CState Vergessen cmsgs)))>

```

Wenn eine Commit-Nachricht gesendet wurde, so wurde keine Abort-Nachricht gesendet

```

67 definition inv3 where
68    $\langle \text{inv3 msgs states} \longleftrightarrow$ 
69      $(\exists \text{sender recpt. (sender, Send recpt Commit)} \in \text{msgs}) \longrightarrow$ 
       $\hookrightarrow (\forall \text{sender recpt. (sender, Send recpt Abort)} \notin \text{msgs}) \rangle$ 


---


44 definition inv3 where
45    $\langle \text{inv3 s} \longleftrightarrow$ 
46      $(\exists \text{sender recpt. (sender, recpt, Commit)} \in \text{snd (fst s)})$ 
       $\hookrightarrow \longrightarrow (\forall \text{sender recpt. (sender, recpt, Abort)} \notin \text{snd (fst$ 
       $\hookrightarrow \text{s)}) \rangle$ 

```

```

19 fun valid_event :: "('proc, 'msg) event  $\Rightarrow$  'proc  $\Rightarrow$ 
20      ('proc  $\times$  ('proc, 'msg) send) set  $\Rightarrow$  bool"
       $\hookrightarrow$  where
21   "valid_event Start _ _ = True" |
22   "valid_event (Receive sender msg) proc msgs = ((sender,
       $\hookrightarrow$  Send proc msg)  $\in$  msgs)" |
23   "valid_event _ _ _ = True"
24
25 inductive execute ::
26   "('proc, 'state, 'msg) step_func  $\Rightarrow$  ('proc  $\Rightarrow$  'state)  $\Rightarrow$ 
       $\hookrightarrow$  'proc set  $\Rightarrow$ 
27   ('proc  $\times$  ('proc, 'msg) event) list  $\Rightarrow$ 
28   ('proc  $\times$  ('proc, 'msg) send) set  $\Rightarrow$  ('proc  $\Rightarrow$  'state)  $\Rightarrow$ 
       $\hookrightarrow$  bool" where
29   "execute step init procs [] {} init" |
30   "[execute step init procs events msgs states;
31    proc  $\in$  procs;
32    valid_event event proc msgs;
33    step proc (states proc) event = (new_state, sent);

```

```

69 inductive reachable :: "('a, 's) ioa  $\Rightarrow$  's  $\Rightarrow$  bool" for C ::
     $\hookrightarrow$  "('a, 's) ioa"
70 where
71   reachable_0: "s  $\in$  starts_of C  $\Rightarrow$  reachable C s"
72 | reachable_n: "reachable C s  $\Rightarrow$  (s, a, t)  $\in$  trans_of C  $\Rightarrow$ 
     $\hookrightarrow$  reachable C t"
73
74 definition invariant :: "[('a, 's) ioa, 's  $\Rightarrow$  bool]  $\Rightarrow$  bool"
75   where "invariant A P  $\longleftrightarrow$  ( $\forall$ s. reachable A s  $\rightarrow$  P s)"
76
380 lemma invariantI:
381   assumes " $\bigwedge$ s. s  $\in$  starts_of A  $\Rightarrow$  P s"
382   and " $\bigwedge$ s t a. reachable A s  $\Rightarrow$  P s  $\Rightarrow$  (s, a, t)  $\in$ 
     $\hookrightarrow$  trans_of A  $\rightarrow$  P t"
383   shows "invariant A P"
  
```

```
90 lemma invariant1:  
91     assumes <execute tupac_step ( $\lambda p$ . Initial (init_val p) UNIV  
     $\hookrightarrow$  r) UNIV events msgs' states'>  
92     shows "inv1 msgs' states'"
```

```
369 lemma invariant1:  
370     "invariant (System t (UNIV - {0}) r) inv1"
```

```
93   using assms proof(induction events arbitrary: msgs'  
    ↪   states' r rule: List.rev_induct)  
  
371 proof(induction rule: invariantI)
```

```

94  case Nil
95  then have statesInit: " $\forall p$ . states' p = Initial (init_val p)
     $\hookrightarrow$  UNIV r"
96    using execute_init assms by fast
97  moreover have msgsEmpty: "msgs' = {}"
98    using execute_init Nil by fast
99  ultimately show ?case
100    using inv1_def by auto
  
```

```

372  case (1 s)
373  moreover have "starts_of (System t (UNIV - {0}) r) =
     $\hookrightarrow$  {((CState (CInitial (t 0) UNIV r) {}), {}),  $\lambda p$ . PState
     $\hookrightarrow$  (PInitial (t p)) {}))}"
374    unfolding starts_of_def System_def par_def
     $\hookrightarrow$  Coordinator_def Channel_def Participants_def by auto
375  ultimately have "s = ((CState (CInitial (t 0) UNIV r)
     $\hookrightarrow$  {}, {}),  $\lambda p$ . PState (PInitial (t p)) {}))"
376    by auto
377  then show ?case
378    unfolding inv1_def by simp
  
```

```

102  case (snoc x events)
103  obtain proc event where "x = (proc, event)"
104    by fastforce
105  hence exec: "execute tupac_step (λp. Initial (init_val p)
    ↪ UNIV r) UNIV
106              (events @ [(proc, event)]) msgs' states'"
107  using snoc.premss assms by fast
108  from this obtain msgs states sent new_state
109  where step_rel1: "execute tupac_step (λp. Initial
    ↪ (init_val p) UNIV r) UNIV events msgs states"
110    and step_rel2: "tupac_step proc (states proc) event =
    ↪ (new_state, sent)"
111    and step_rel3: "msgs' = msgs ∪ ((λmsg. (proc, msg)) `
    ↪ sent)"
112    and step_rel4: "states' = states (proc := new_state)"
113  by auto
  
```

```

380  case (2 s t a)
381  then show ?case
382  using invariant1_trans by metis
  
```

```

114 show ?case
115 proof (cases "proc = 0")
116   case True
117   then have "coordinator_step proc (states 0) event =
    ↪ (new_state, sent)"
118     using step_rel2 by simp
119   moreover have "inv1 msgs states"
120     using snoc.IH step_rel1 by fast
121   ultimately show ?thesis
122     using invariant1_coordinator True step_rel3 step_rel4
    ↪ exec by metis
  
```

```

380 case (2 s t a)
381 then show ?case
382   using invariant1_trans by metis
  
```

```

124 case False
125 then have "participant_step (states proc) event =
    ↪ (new_state, sent)"
126   using step_rel2 by simp
127 moreover have "inv1 msgs states"
128   using snoc.IH step_rel1 by fast
129 ultimately show ?thesis
130   using invariant1_participant False step_rel3 step_rel4
    ↪ exec by metis
  
```

qed
 qed

```

380 case (2 s t a)
381 then show ?case
382   using invariant1_trans by metis
  
```

```

5 lemma invariant1_coordinator:
6   assumes "coordinator_step 0 (states 0) event = (new_state,
    ↪ sent)"
7   and "msgs' = msgs  $\cup$  (( $\lambda$ msg. (0, msg)) ` sent)"
8   and "states' = states (0 := new_state)"
9   and "execute tupac_step ( $\lambda$ p. Initial (init_val p) UNIV r)
    ↪ UNIV (events @ [(0, event)]) msgs' states'"
10  and "inv1 msgs states"
11  shows "inv1 msgs' states'"
  
```

```

63 lemma invariant1_trans:
64   assumes "reachable (System t (UNIV - {0}) r) s"
65   and "inv1 s"
66   and "(s, a, s')  $\in$  trans_of (System t (UNIV - {0}) r)"
67   shows "inv1 s'"
  
```

```
12  using assms apply (induction rule: coordinator_induct)
13  using inv1_def UnCI assms(2,3,5) apply (metis (mono_tags,
    ↪ lifting) fun_upd_apply)+
14  done
```

```
68  proof -
```

```

16 lemma invariant1_participant:
17   assumes "participant_step (states proc) event =
      ↪ (new_state, sent)"
18   and "proc ≠ 0"
19   and "msgs' = msgs ∪ ((λmsg. (proc, msg)) ` sent)"
20   and "states' = states (proc := new_state)"
21   and "execute tupac_step (λp. Initial (init_val p) UNIV r)
      ↪ UNIV (events @ [(proc, event)]) msgs' states'"
22   and "inv1 msgs states"
23   shows "inv1 msgs' states'"
24   using assms
25   proof (induction rule: participant_induct)

```

```

63 lemma invariant1_trans:
64   assumes "reachable (System t (UNIV - {0}) r) s"
65   and "inv1 s"
66   and "(s, a, s') ∈ trans_of (System t (UNIV - {0}) r)"
67   shows "inv1 s'"
68   proof -

```

25 **proof** (induction rule: participant_induct)

```

69  obtain cstate cmsgs chmsgs pstates where decomp_s: "s =
    ↪ ((CState cstate cmsgs, chmsgs), pstates)"
70    by (metis coord_state.exhaust surj_pair)
71  obtain cstate' cmsgs' chmsgs' pstates' where decomp_s': "s'
    ↪ = ((CState cstate' cmsgs', chmsgs'), pstates')"
72    by (metis coord_state.exhaust surj_pair)
  
```



```
25  proof (induction rule: participant_induct)
```

```
73  show ?thesis  
74    using assms(3) unfolding decomp_s decomp_s'  
75  proof (induction rule: System_trans_inv)  
76    case System_Step  
77    then show ?case  
78    proof(cases a)
```

25 **proof** (induction rule: participant_induct)

79 **case** (Start rcpt)
80 **then have** "pstates = pstates'"
81 **using** System_Step(3) **unfolding** parts_trans_def **by** auto
82 **moreover have** "chmsgs = chmsgs'"
83 **using** Start System_Step(2) **by** auto

```
25  proof (induction rule: participant_induct)

84      ultimately have "inv1 s  $\longleftrightarrow$  inv1 s'"
85      using inv1_def by (metis assms(2) decomp_s decomp_s'
       $\hookrightarrow$  fst_conv snd_conv)
86  then show ?thesis
87      using assms(2) decomp_s decomp_s' by simp
```

25 **proof** (induction rule: participant_induct)

89 **case** (Send sender rcpt _)
 90 **then have** "($\exists$ msgs p. pstates p = PState PCommitted msgs)
 $\leftrightarrow$ $\longleftrightarrow$ ($\exists$ msgs p. pstates' p = PState PCommitted msgs)"
 91 **by** (metis System_Step.hyps(3) part_state.exhaust
 $\leftrightarrow$ parts_trans_Send)

25 **proof** (induction rule: participant_induct)

92 **moreover have** " $(\exists \text{sender rcpt. (sender, rcpt, Commit)} \in$
 $\hookrightarrow \text{chmsgs}) \longrightarrow (\exists \text{sender rcpt. (sender, rcpt, Commit)} \in$
 $\hookrightarrow \text{chmsgs}')$ "
 93 **using** Send System_Step(2) **by** auto
 94 **ultimately have** " $\text{inv1 } s \longleftrightarrow \text{inv1 } s'$ "
 95 **using** inv1_def **by** (metis assms(2) decomp_s decomp_s'
 $\hookrightarrow \text{fst_conv snd_conv}$)
 96 **then show** ?thesis
 97 **using** assms(2) decomp_s decomp_s' **by** simp

25 **proof** (induction rule: participant_induct)

```

99   case (Receive sender rcpt msg)
100  have chEquiv:"chmsgs = chmsgs'"
101    using Receive System_Step(2) by blast
102  moreover have receive:"(sender, rcpt, msg) ∈ chmsgs'"
103    by (metis Receive decomp_s decomp_s'
104      ↪ System_Receive_chan_step assms(3))
104  then show ?thesis
105  proof(cases "rcpt = 0")
  
```

25 **proof** (induction rule: participant_induct)

```

106   case True
107   then have "pstates = pstates'"
108     using True System_Step(3) Receive unfolding
       $\hookrightarrow$  parts_trans_def by auto
109   then have "inv1 s  $\longleftrightarrow$  inv1 s'"
110     using inv1_def chEquiv by (metis decomp_s decomp_s'
       $\hookrightarrow$  fst_conv snd_conv)
111   then show ?thesis
112     using assms(2) decomp_s decomp_s' by simp
  
```

```
25  proof (induction rule: participant_induct)

114      case False
115      then have otherPartsEquiv: " $\forall p \neq rcpt. pstates\ p =$   
       $\hookrightarrow pstates'\ p$ "
116      using System_Step(3) Receive unfolding  
       $\hookrightarrow$  parts_trans_def by auto
```


25 **proof** (induction rule: participant_induct)

```

118   proof(cases "pstates rcpt = pstates' rcpt")
119     case True
120     then have "pstates = pstates'"
121       using otherPartsEquiv by auto
122     then show ?thesis
123       using chEquiv assms(2) inv1_def decomp_s
        ↪ decomp_s' by (metis fst_conv snd_conv)
  
```

25 **proof** (induction rule: participant_induct)

```

125   case False
126   then obtain pstate pmsgs where stateDecomp: "pstates
    ↪ rcpt = PState pstate pmsgs"
127     using part_state.exhaust by blast
128   then obtain new_state sent where
    ↪ step: "participant_step rcpt pstate a =
    ↪ (new_state, sent)"
129     using System_Step(3) False by fastforce
130   have update_eq: "pstates' rcpt = PState new_state
    ↪ (pmsgs ∪ ((λmsg. (rcpt, msg)) ` sent))"
131   by (smt (verit, ccfv_SIG) step parts_trans_proj
    ↪ stateDecomp parts_trans_step_result
    ↪ System_Step(3) False Receive)
  
```